State-Compute Replication: Parallelizing High-Speed Stateful Packet Processing

Qiongwen Xu, Sebastiano Miano, Xiangyu Gao, Tao Wang, Adithya Murugadass, Songyuan Zhang, Anirudh Sivaraman, Gianni Antichi, Srinivas Narayana

INTRODUCTION

- Growing gap between NIC speed and CPU performance
- Multi-core processing needed for high-speed networking
- **Flow**: packets accessing the same *state*
- **State**: per-flow data maintained by network function
- Stateful packets --> require consistent state updates
- Stateless packets --> easily parallelizable
- Same-flow packets must be processed consistently

Who cares about High-Speed Packet Processing?

- Middleboxes (NFs -connection trackers,firewalls, etc)
- High-volume compute-light apps - stream based logging systems

BACKGROUND & OUR GOALS

The two main techniques used to parallelize such programs across cores are:

1. **Shared State Parallelism:** Guarding access to shared memory using synchronization logic. Only works well when:
   a. packets of a single flow arrive slowly
   b. efficient implementations of synchronization are available
   But, neither of above is generally applicable.

- Performance degrades due to synchronization overhead once additional cores are introduced

2. **Sharded parallelism:**
   a. Sharding is achieved through techniques like Receive Side Scaling (RSS), available on modern NICs.
   b. Packets are dispatched to the cores based on flow. This eliminates synchronization by partitioning state.

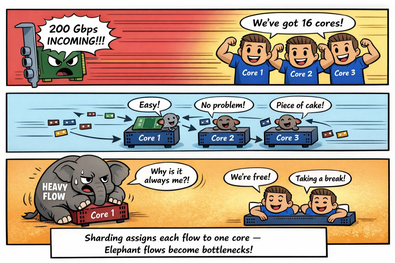


Illustration of the problem scenario in sharding

- This may create load imbalance across the cores if some flows are heavier than others. *(Elephant flows)*

- Goal is to eliminate contention and skew-dependent bottlenecks by replicating states and computation across the cores

METHODOLOGY

1. **Packet history sequencer:**
   - Used to steer packets across the cores
   - Maintains the most recent packet history
   - Attaches the history to each packet sent to the cores

2. **Performance Gain:**
   - Only state computation is replicated for each missing packet of the flow on each core
   - Dispatch and parsing are required only once per packet

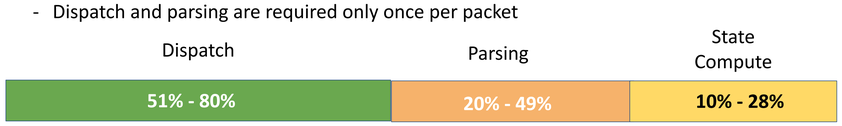
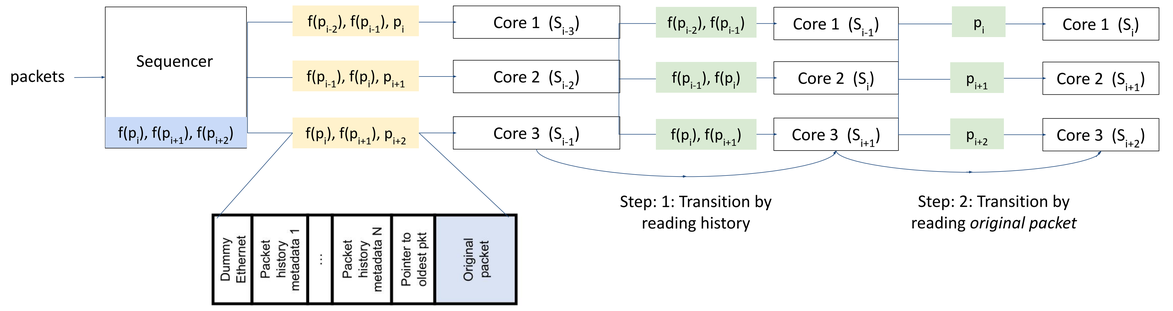


Fig. 1(b): Packet time processing breakdown (XDP) of single packet

Fig. 1(a): An example illustrating the SCR sequencer. P_j is the ith packet received by the sequencer, $f(P_j)$ are relevant fields from p_j, and S_i is the state after processing packets $p_1, \ldots, p_i$ in order



Step: 1: Transition by reading history

Step: 2: Transition by reading *original packet*

EVALUATION

Experimental setup:

- Two server machines connected back-to-back with one serving as the packet generator
- NIC: 100 Gbps Nvidia/Mellanox ConnectX-5
- CPU/RAM: Intel Xeon Gold 6334 (16 cores) / 256GB DDR4 (2 NUMA nodes)

Programs	Traces	Compared
a) Heavy-hitter monitor b) DDoS Mitigator c) Token bucket policer d) Port-knocking firewall	a) University DC b) CAIDA c) Hyperscalar DC	a) State sharing approach b) State sharding using RSS c) Sharding using RSS++

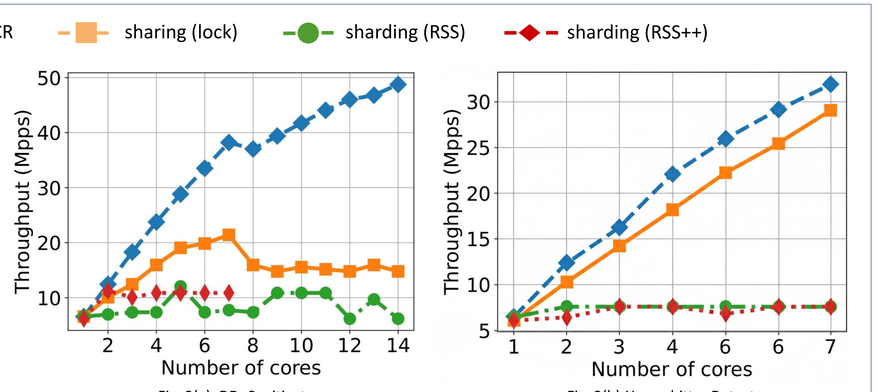
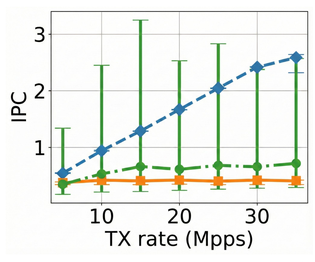


Fig. 2(a): DDoS mitigator

Fig. 2(b):Heavy-hitter Detector

WHY SCR SCALES?



- **IPC increases** with the offered load
- CPU gets busier with packet processing
- Eliminates the lock contention
- No inter-core synchronization
- **Lowers L2 miss** ratio
- For sharding approaches, IPC values drop significantly with additional cores.

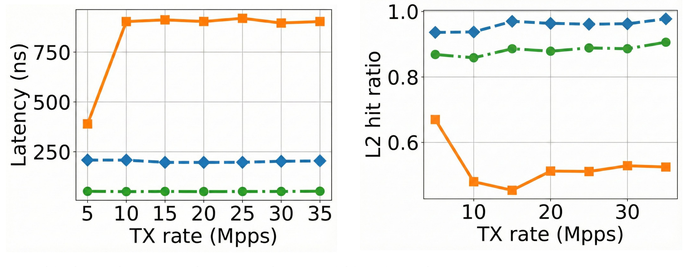


Fig. 3: Hardware performance metrics while executing token bucket program. This shows the program's compute latency, L2 hit ratio, and instructions per clock cycle (IPC), when program is scaled to 7 cores.

LIMITATIONS

Limits for Scaling:

1. As the compute latency increases relative to dispatch latency, the effectiveness of the SCR reduces.
2. Attaching histories to packets incurs additional byte-per-second overhead. This increases L3 cache occupancy and incurs additional PCIe transactions.

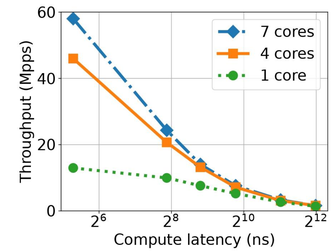
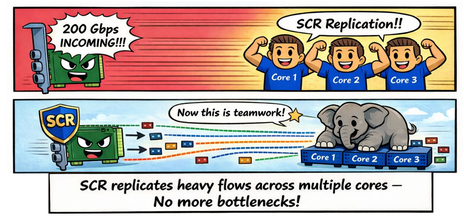


Fig.4: Throughput scaling of a stateless program using SCR, as compute latency varies

- If the path between the sequencer and cores is lossy, it creates cross-core state inconsistency. Additional operations for loss recovery add extra overhead.

CONCLUSION

- Stateful packet processing programs do not scale with traditional approaches like sharding and shared-state parallelism with increasing cores.
- SCR enables this scaling by replicating state and computation across the cores and preserves correctness.



REFERENCES

- State-compute replication: parallelizing high-speed stateful packet processing. In Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25). USENIX Association, USA, Article 50, 937–957.

- Comic source: Generated using AI (ChatGPT).

Scan QR code to see detailed references and download the paper